

Generator funkcyjny z wykorzystaniem DDS dla układów FPGA

Dokumentacja techniczna

Filip Izworski

15.06.2026

Spis treści

Lista oznaczeń	3
1. Wstęp	4
1.1 Cel projektu	4
1.2 Zakres projektu	4
2. Funkcjonalność	5
2.1 Obsługiwane przebiegi	5
2.2 Konfiguracja nośnej	5
2.3 Konfiguracja modulacji	5
2.4 Interfejs wejścia-wyjścia	6
3. Analiza problemu	7
3.1 Synteza cyfrowa DDS	7
3.2 Regulacja częstotliwości	7
3.3 Akumulator fazy	7
3.4 Generacja przebiegów	7
3.5 Modulacja sygnału	8
3.5.1 Modulacja amplitudy	8
3.5.2 Modulacja częstotliwości	8
3.5.3 Modulacja fazy	8
3.6 Arytmetyka stałoprzecinkowa	8
4. Projekt techniczny	9
4.1 Architektura systemu	9
4.2 Konfiguracja generatora	9
4.3 Przepływ danych	10
4.4 Tor generacji sygnału DDS	10
4.5 Tor modulacji	12
5. Opis realizacji	14
5.1 Organizacja projektu	14
5.2 Konfiguracja systemu	14
5.3 Implementacja generatora DDS	14
5.3.1 Akumulator fazy	14
5.3.2 Sterowanie fazą	14
5.3.3 Generacja przebiegów	14
5.4 Implementacja modulacji	15
5.4.1 Modulacja amplitudy	15
5.4.2 Modulacja częstotliwości	15
5.4.3 Modulacja fazy	15
5.5 Arytmetyka stałoprzecinkowa	15
5.6 Najważniejsze decyzje projektowe	16
5.7 Ograniczenia implementacji	17
6. Opis wykonanych testów	18
6.1 Cel testów	18
6.2 Środowisko testowe	18
6.3 Testy funkcjonalne	18
6.4 Analiza przebiegów	19

6.4.1	Przebieg sinusoidalny	19
6.4.2	Przebieg prostokątny	19
6.4.3	Przebieg trójkątny	20
6.4.4	Przebieg AM	20
6.4.5	Przebieg FM	20
6.4.6	Przebieg PM	20
6.5	Porównanie z modelem Octave	20
7.	Podręcznik użytkownika	22
7.1	Wymagane narzędzia	22
7.2	Struktura projektu	22
7.3	Symulacja z użyciem Verilator	22
7.4	Użycie modelu Octave	23
7.5	Użycie w Vivado	23
7.6	Konfiguracja generatora DDS	23
8.	Metodologia rozwoju i utrzymania systemu	24
8.1	Organizacja projektu	24
8.2	Metodologia weryfikacji	24
8.3	Model referencyjny Octave	24
8.4	Możliwości dalszego rozwoju	24

Lista oznaczeń

Oznaczenie	Znaczenie
DDS	Direct Digital Synthesis
FPGA	Field Programmable Gate Array
RTL	Register Transfer Level
FTW	Frequency Tuning Word
AM	Amplitude Modulation
FM	Frequency Modulation
PM	Phase Modulation

1. Wstęp

1.1 Cel projektu

Celem projektu jest implementacja generatora funkcyjnego z wykorzystaniem metody DDS (Direct Digital Synthesis) w języku opisu sprzętu. Generator umożliwia generację przebiegów okresowych o konfigurowalnej częstotliwości, fazie i amplitudzie oraz obsługuje modulację AM, FM i PM. Projekt został przygotowany z myślą o implementacji w układach FPGA jako osobny moduł RTL lub blok IP większego systemu.

1.2 Zakres projektu

Zakres projektu obejmuje:

- model referencyjny w Octave,
- wykorzystanie języka SystemVerilog do opisu RTL i testbenchy,
- pakiet definicji i parametrów `dds_pkg`,
- główny moduł generatora funkcyjnego `dds_core`,
- moduł integracyjny dla Vivado `dds_core_wrapper`,
- akumulator fazy oraz sterowanie fazą,
- generację przebiegów DC, sinusoidalnego, prostokątnego i trójkątnego,
- modulację AM, FM i PM,
- skalowanie amplitudy sygnału wyjściowego,
- testbenche dla wszystkich większych modułów.

Projekt nie obejmuje implementacji kompletnego interfejsu rejestrowego ani integracji z konkretną platformą sprzętową.

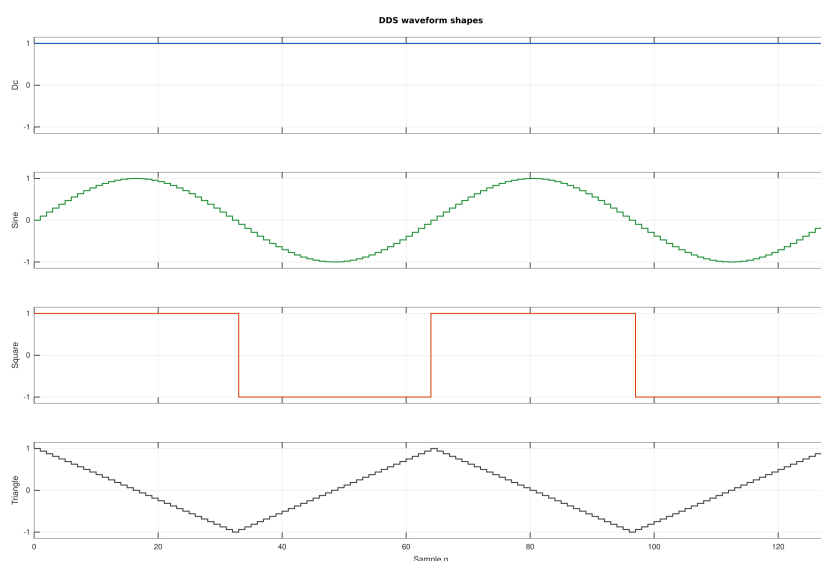
2. Funkcjonalność

2.1 Obsługiwane przebiegi

Generator obsługuje następujące typy przebiegów:

- WAVE_DC – wartość stała,
- WAVE_SINE – przebieg sinusoidalny,
- WAVE_SQUARE – przebieg prostokątny,
- WAVE_TRIANGLE – przebieg trójkątny.

Rysunek 2.1 przedstawia przebiegi wygenerowane przez model referencyjny Octave.



Rysunek 2.1: Przebiegi obsługiwane przez generator DDS.

2.2 Konfiguracja nośnej

Parametry nośnej zestawiono w tabeli 2.1.

Tablica 2.1: Parametry konfiguracji nośnej

Pole	Opis
carr_ftw	Słowo strojenia częstotliwości (FTW).
carr_phase	Stałe przesunięcie fazy.
carr_amp	Amplituda nośnej.
carr_wave	Typ generowanego przebiegu.

2.3 Konfiguracja modulacji

Generator obsługuje następujące tryby pracy:

- MOD_NONE – brak modulacji,
- MOD_AM – modulacja amplitudy,

- MOD_FM – modulacja częstotliwości,
- MOD_PM – modulacja fazy.

Parametry modulacji przedstawiono w tabeli 2.2.

Tablica 2.2: Parametry konfiguracji modulacji

Pole	Opis
<code>mod_ftw</code>	Częstotliwość sygnału modulującego.
<code>mod_depth</code>	Głębokość modulacji.
<code>mod_type</code>	Typ modulacji.
<code>mod_wave</code>	Typ przebiegu modulującego.

2.4 Interfejs wejścia-wyjścia

Na wejście generatora oprócz sygnałów zegarowego i resetu, przekazywane są sygnały konfiguracyjne. Są one przekazywane przez strukturę `dds_config_t` (SystemVerilog) lub przez porty modułu `dds_core_wrapper` (Verilog).

Wyjściem generatora jest próbka `o_sample` o szerokości określonej parametrem `SAMPLE_WIDTH`. To czy próbka jest poprawna i zdalna do użycia, jest sygnalizowane przez sygnał `o_sample_valid`.

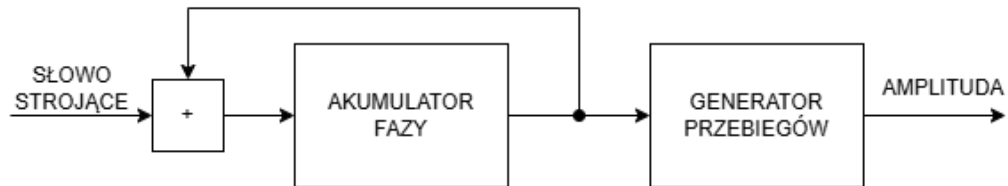
Ze względu na potokową implementację generatora zmiany konfiguracji nie są widoczne natychmiast. Odbiór próbek powinien być zsynchronizowany z sygnałem `o_sample_valid`.

3. Analiza problemu

3.1 Synteza cyfrowa DDS

Direct Digital Synthesis (DDS) jest metodą cyfrowej generacji przebiegów okresowych. Podstawowa idea polega na reprezentowaniu położenia w okresie sygnału za pomocą cyfrowej fazy, która jest zwiększana o pewną wartość w każdym cyklu zegara. Aktualna wartość fazy jest następnie przekształcana na odpowiadającą jej wartość amplitudy.

Podstawowy tor DDS składa się z akumulatora fazy oraz generatora przebiegu wykorzystującego aktualną wartość fazy. Uproszczony schemat działania przedstawiono na rysunku 3.1.



Rysunek 3.1: Zasada działania generatora DDS.

3.2 Regulacja częstotliwości

Częstotliwość generowanego sygnału jest określana przez słowo strojenia częstotliwości (Frequency Tuning Word, FTW), które definiuje przyrost fazy wykonywany podczas każdego cyklu zegara.

Dla częstotliwości zegara f_{clk} oraz akumulatora fazy o szerokości N bitów częstotliwość sygnału wyjściowego można opisać zależnością:

$$f_{out} = \frac{FTW}{2^N} \cdot f_{clk} \quad (3.1)$$

Odwrotnie, wymagane słowo FTW można wyznaczyć ze wzoru:

$$FTW = \text{round} \left(\frac{f_{out}}{f_{clk}} \cdot 2^N \right) \quad (3.2)$$

Rozdzielczość częstotliwościowa generatora DDS wynosi:

$$\Delta f = \frac{f_{clk}}{2^N} \quad (3.3)$$

Zwiększenie szerokości akumulatora fazy poprawia rozdzielczość częstotliwościową, kosztem większego zużycia zasobów sprzętowych.

3.3 Akumulator fazy

Akumulator fazy przechowuje aktualne położenie w okresie generowanego sygnału. Jego działanie opisuje równanie:

$$phase[n + 1] = (phase[n] + FTW) \bmod 2^N \quad (3.4)$$

Operacja modulo powoduje cykliczne zawijanie fazy po osiągnięciu końca zakresu reprezentacji. Dzięki temu możliwe jest generowanie sygnałów okresowych przy użyciu prostych operacji dodawania.

3.4 Generacja przebiegów

Generator przebiegu przekształca aktualną wartość fazy na wartość amplitudy sygnału. Sygnał może być generowany za pomocą gotowych próbek umieszczonych w pamięci LUT lub z wykorzystaniem bardziej skomplikowanych algorytmów, na przykład CORDIC. W zależności od zastosowanej metody możliwa jest

generacja różnych typów przebiegów okresowych, takich jak sygnały sinusoidalne, prostokątne czy trójkątne.

Przykładowe przebiegi przedstawiono na rysunku [2.1](#).

3.5 Modulacja sygnału

W wielu zastosowaniach wymagane jest dynamiczne modyfikowanie parametrów generowanego sygnału. Najczęściej stosowane są modulacja amplitudy (AM), częstotliwości (FM) oraz fazy (PM).

3.5.1 Modulacja amplitudy

Modulacja amplitudy polega na zmianie amplitudy sygnału nośnego zgodnie z wartością sygnału modulującego. Parametry częstotliwości i fazy pozostają niezmienione.

3.5.2 Modulacja częstotliwości

Modulacja częstotliwości polega na zmianie chwilowej częstotliwości sygnału nośnego. W generatorach DDS realizuje się ją poprzez modyfikację efektywnego przyrostu fazy.

3.5.3 Modulacja fazy

Modulacja fazy polega na zmianie chwilowej wartości fazy sygnału przy zachowaniu niezmienionej częstotliwości średniej.

3.6 Arytmetyka stałoprzecinkowa

Układy FPGA realizują obliczenia na słowach o skończonej szerokości. Z tego powodu w praktycznych implementacjach DDS stosowana jest arytmetyka stałoprzecinkowa zamiast zmiennoprzecinkowej.

Ograniczona liczba bitów wpływa na:

- rozdzielczość częstotliwości,
- dokładność reprezentacji amplitudy,
- błędy kwantyzacji,
- konieczność zaokrąglania i saturacji wyników.

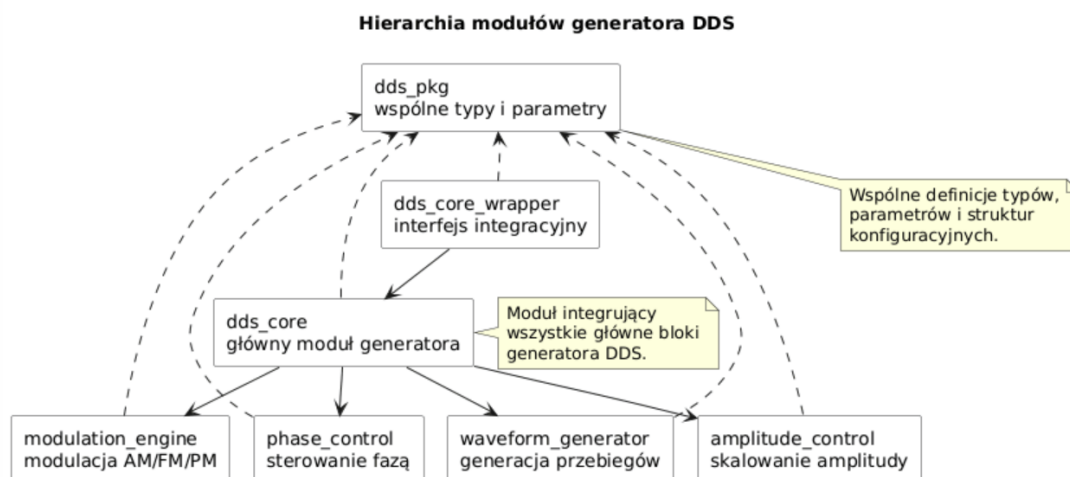
Dobór szerokości poszczególnych ścieżek danych stanowi kompromis pomiędzy dokładnością generowanego sygnału a wykorzystaniem zasobów sprzętowych.

4. Projekt techniczny

4.1 Architektura systemu

Generator został zaprojektowany jako zestaw współpracujących modułów realizujących konfigurację, generację sygnału oraz modulację. Głównym blokiem systemu jest moduł `dds_core`, natomiast `dds_core_wrapper` udostępnia uproszczony interfejs przeznaczony do integracji z większym systemem FPGA.

Hierarchię modułów przedstawiono na rysunku 4.1.



Rysunek 4.1: Hierarchia modułów generatora DDS.

Architektura została podzielona na trzy główne części:

- konfigurację generatora,
- tor generacji sygnału DDS,
- tor modulacji.

Taki podział pozwala na niezależne rozwijanie i testowanie poszczególnych bloków systemu.

4.2 Konfiguracja generatora

Konfiguracja pracy generatora jest przechowywana w strukturze `dds_config_t`. Zawiera ona parametry nośnej oraz ustawienia modulacji.

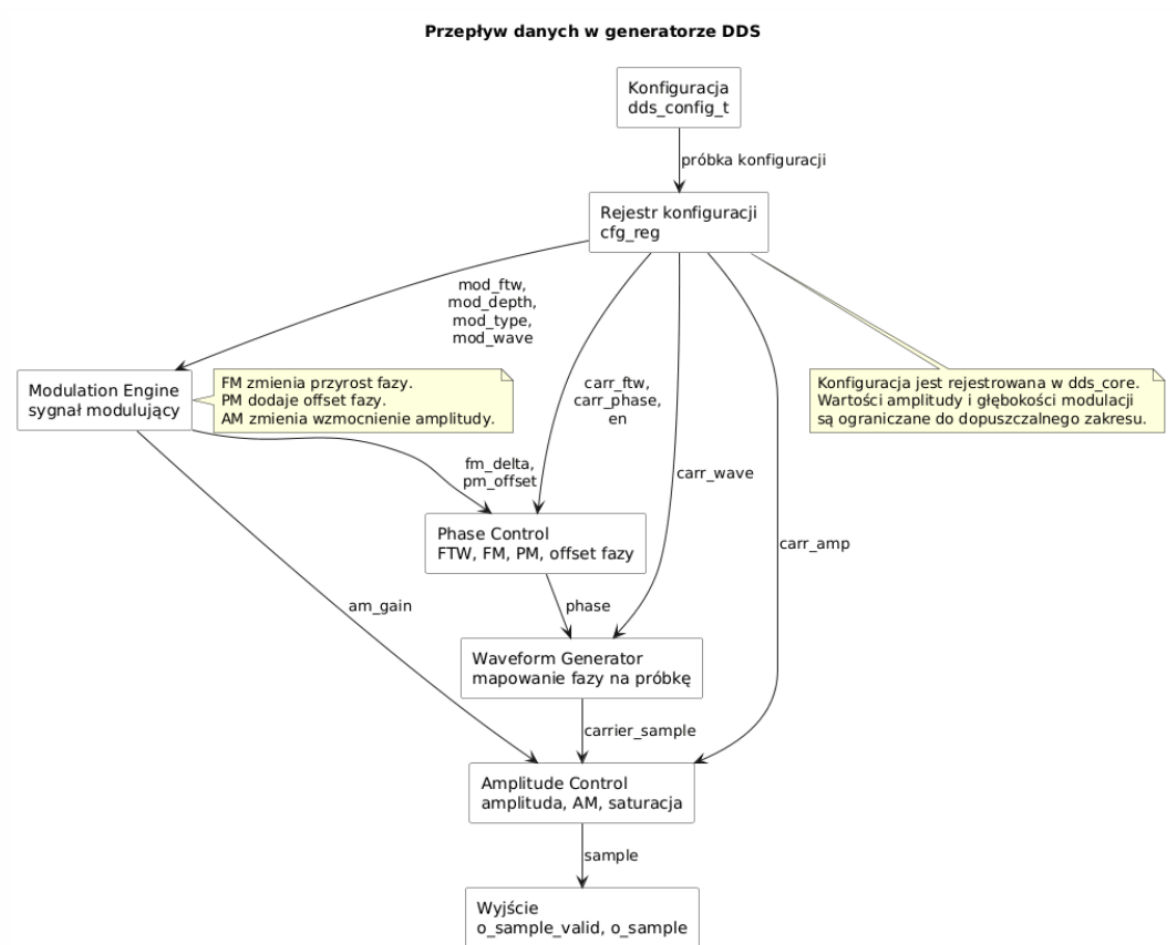
Tablica 4.1: Pola struktury `dds_config_t`

Pole	Opis
<code>en</code>	Włączenie generatora.
<code>rst</code>	Reset toru sygnałowego.
<code>carr_ftw</code>	Częstotliwość nośnej.
<code>carr_phase</code>	Stałe przesunięcie fazy nośnej.
<code>carr_amp</code>	Amplituda nośnej.
<code>carr_wave</code>	Typ przebiegu nośnej.
<code>mod_ftw</code>	Częstotliwość sygnału modulującego.
<code>mod_depth</code>	Głębokość modulacji.
<code>mod_type</code>	Typ modulacji.
<code>mod_wave</code>	Typ przebiegu modulującego.

Moduł `dds_core` rejestruje konfigurację i udostępnia ją pozostałym blokom systemu.

4.3 Przepływ danych

Przepływ danych w generatorze przedstawiono na rysunku 4.2.



Rysunek 4.2: Przepływ danych w generatorze DDS.

Przetwarzanie sygnału odbywa się w następujących etapach:

1. odczyt konfiguracji,
2. wyznaczenie parametrów modulacji,
3. obliczenie fazy nośnej,
4. generacja próbki przebiegu,
5. skalowanie amplitudy,
6. wygenerowanie próbki wyjściowej.

Poszczególne etapy są realizowane przez niezależne moduły połączone w strukturę potokową.

4.4 Tor generacji sygnału DDS

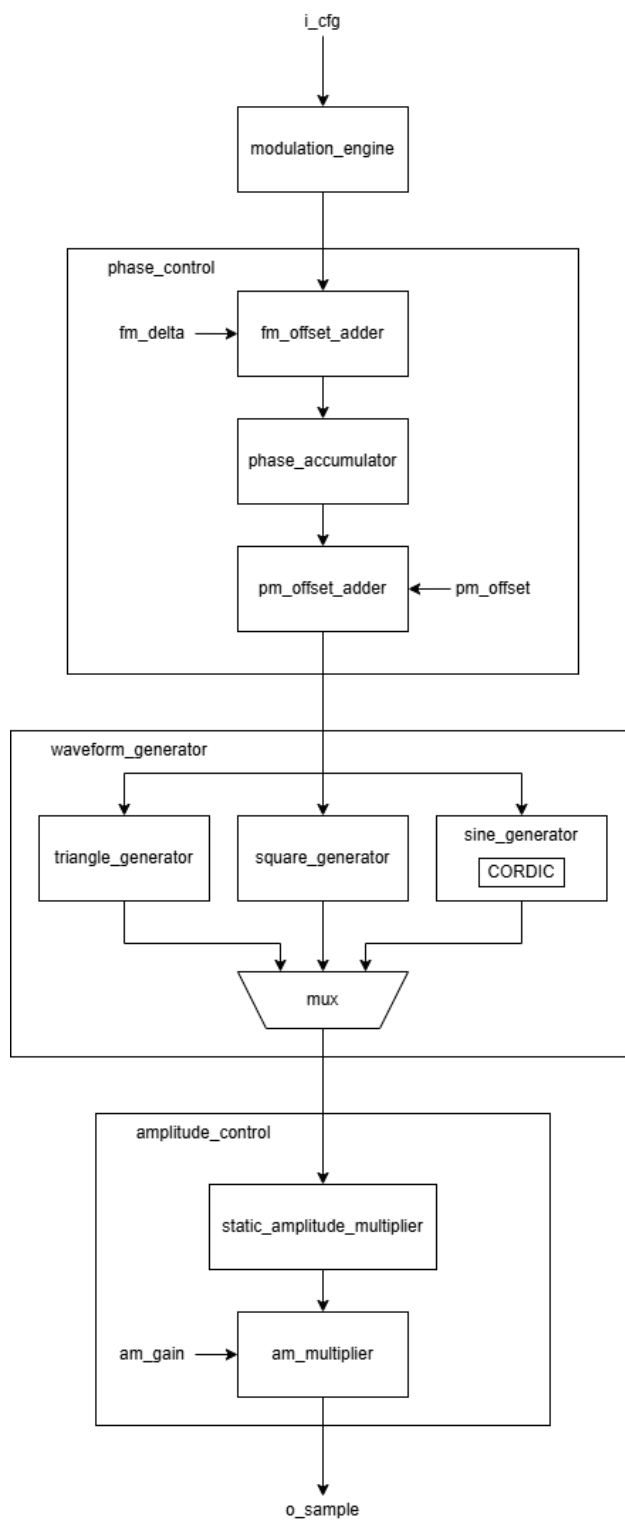
Tor generacji sygnału odpowiada za przekształcenie parametrów nośnej na próbki sygnału wyjściowego. Jego podstawowymi elementami są:

- blok sterowania fazą,
- akumulator fazy,
- generator przebiegów,

- blok regulacji amplitudy.

Wartość częstotliwości nośnej jest reprezentowana przez słowo FTW, które steruje przyrostem fazy w akumulatorze. Aktualna faza jest następnie zamieniana na próbkę wybranego przebiegu. Ostatnim etapem jest skalowanie amplitudy oraz przygotowanie próbki wyjściowej.

Diagram blokowy generatora przedstawiono na rysunku 4.3.



Rysunek 4.3: Diagram blokowy generatora DDS.

4.5 Tor modulacji

Modulacja realizowana jest przez dedykowany blok generujący sygnał modulujący niezależnie od toru nośnej.

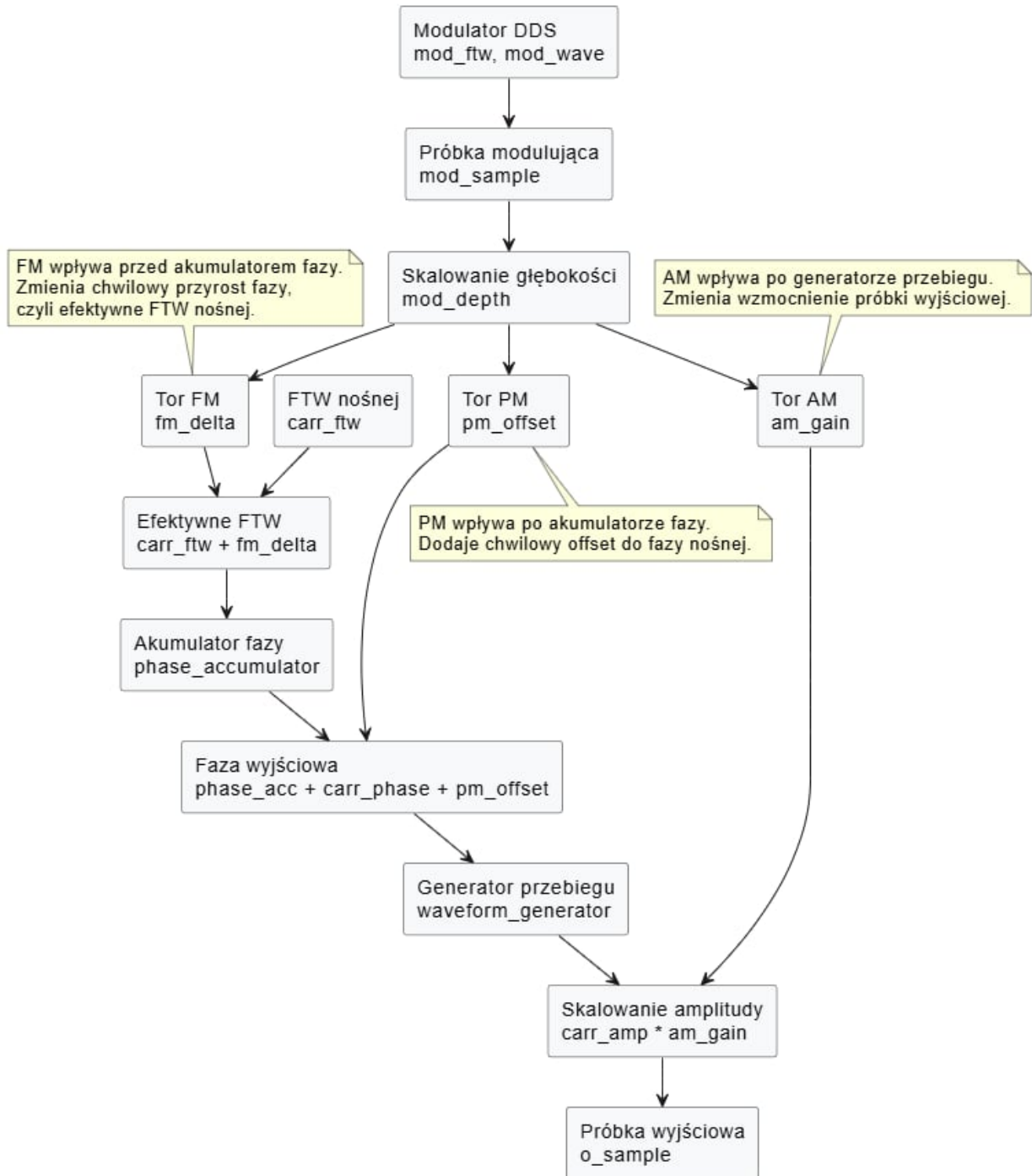
Modulator wykorzystuje własne parametry częstotliwości i kształtu przebiegu, dzięki czemu możliwe jest niezależne sterowanie sygnałem modulującym.

W zależności od wybranego trybu pracy sygnał modulujący wpływa na:

- amplitudę nośnej (AM),
- częstotliwość nośnej (FM),
- fazę nośnej (PM).

Zależności pomiędzy poszczególnymi torami przedstawiono na rysunku [4.4](#).

Wpływ modulacji AM/FM/PM na tor DDS



Rysunek 4.4: Integracja modulacji AM, FM i PM z generatorem DDS.

5. Opis realizacji

5.1 Organizacja projektu

Projekt został podzielony na cztery główne części:

- kod RTL w języku SystemVerilog,
- środowisko symulacyjne i testbenche,
- model referencyjny Octave.

Wspólne definicje typów, parametrów oraz struktur konfiguracyjnych zostały umieszczone w pakiecie `dds_pkg`. Pozostałe moduły korzystają z tych samych definicji, co upraszcza utrzymanie spójności projektu.

5.2 Konfiguracja systemu

Parametry pracy generatora są przechowywane w strukturze `dds_config_t`. Zawiera ona konfigurację nośnej, ustawienia modulacji oraz sygnały sterujące pracą generatora.

Moduł `dds_core` rejestruje konfigurację i udostępnia ją pozostałym blokom systemu. Dodatkowo wartości amplitudy oraz głębokości modulacji są ograniczane do dopuszczalnego zakresu przed wykorzystaniem w dalszych obliczeniach.

5.3 Implementacja generatora DDS

5.3.1 Akumulator fazy

Moduł `phase_accumulator` realizuje podstawowy mechanizm DDS. W każdym cyklu zegara aktualna wartość fazy jest zwiększana o wartość FTW. Przepelnienie rejestru jest wykorzystywane jako naturalna implementacja operacji modulo.

Ten sam moduł został wykorzystany zarówno w torze nośnej, jak i w generatorze sygnału modulującego, co pozwoliło ograniczyć liczbę różnych bloków funkcjonalnych.

5.3.2 Sterowanie fazą

Moduł `phase_control` odpowiada za wyznaczenie końcowej fazy przekazywanej do generatora przebiegu.

Blok uwzględnia:

- częstotliwość nośnej określoną przez `carr_ftw`,
- statyczne przesunięcie fazowe `carr_phase`,
- modulację FM,
- modulację PM.

W przypadku modulacji częstotliwości efektywne FTW jest modyfikowane przez wartość `fm_delta`. Modulacja fazy realizowana jest przez dodanie zmiennego offsetu `pm_offset` do fazy nośnej.

5.3.3 Generacja przebiegów

Moduł `waveform_generator` odpowiada za zamianę fazy na próbkę amplitudy.

Obsługiwane są cztery typy przebiegów:

- `WAVE_DC`,
- `WAVE_SINE`,
- `WAVE_SQUARE`,

- WAVE_TRIANGLE.

Przebieg sinusoidalny jest generowany przy użyciu algorytmu CORDIC, natomiast przebiegi prostokątny i trójkątny są wyznaczane bezpośrednio na podstawie aktualnej wartości fazy.

Ponieważ poszczególne ścieżki charakteryzują się różną latencją, generator zawiera dodatkowe rejestry wyrównujące opóźnienia sygnałów.

5.4 Implementacja modulacji

Modulacja została zrealizowana w module `modulation_engine`, który wykorzystuje własny generator DDS do wytworzenia sygnału modulującego.

Dzięki temu częstotliwość oraz kształt sygnału modulującego mogą być konfigurowane niezależnie od nośnej.

5.4.1 Modulacja amplitudy

Dla modulacji AM sygnał modulujący jest przekształcany na współczynnik wzmocnienia `am_gain`. Współczynnik ten jest następnie wykorzystywany podczas skalowania amplitudy nośnej.

Przyjęto założenie, że amplituda nie może przyjmować wartości ujemnych, dlatego wartości wykraczające poza dopuszczalny zakres są ograniczane.

5.4.2 Modulacja częstotliwości

W trybie FM sygnał modulujący jest przekształcany na wartość `fm_delta`, która modyfikuje efektywne FTW nośnej.

Wprowadzono dodatkowe ograniczenia zabezpieczające przed wyjściem poza dopuszczalny zakres częstotliwości.

5.4.3 Modulacja fazy

Dla modulacji PM sygnał modulujący jest przeliczany na wartość `pm_offset`. Offset jest dodawany do fazy nośnej po akumulacji fazy, dzięki czemu modulacja wpływa na chwilowe położenie fazowe sygnału bez zmiany jego średniej częstotliwości.

5.5 Arytmetyka stałoprzecinkowa

Projekt wykorzystuje wyłącznie arytmetykę stałoprzecinkową.

Faza reprezentowana jest jako liczba bez znaku, natomiast próbki sygnałów jako liczby ze znakiem. Operacje mnożenia powodują czasowe rozszerzenie szerokości słów, co pozwala ograniczyć błędy kwantyzacji.

Szczególną uwagę poświęcono torowi amplitudy, w którym wykonywane są:

- skalowanie przez amplitudę nośnej,
- skalowanie przez współczynnik `am_gain`,
- redukcja szerokości słowa,
- zaokrąglanie,
- saturacja wyniku.

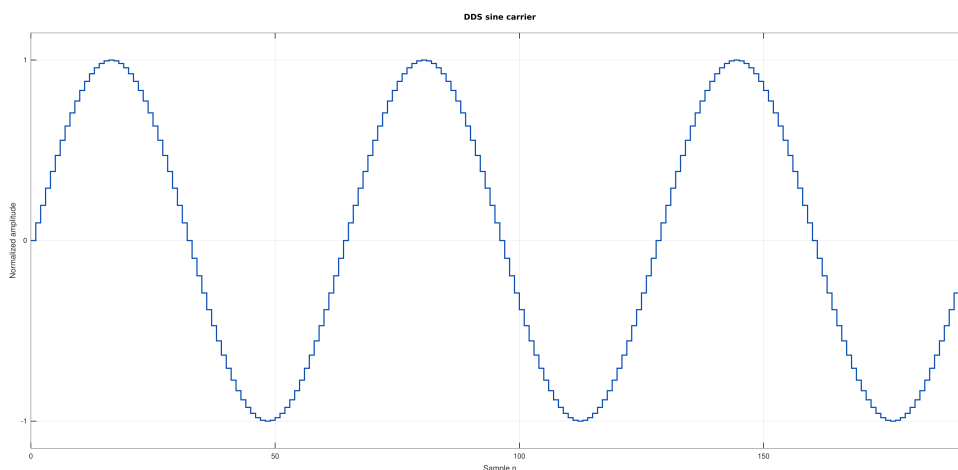
Zaokrąglanie ogranicza błędy kwantyzacji, natomiast saturacja zapobiega zawijaniu wyniku po przekroczeniu dopuszczalnego zakresu wartości.

5.6 Najważniejsze decyzje projektowe

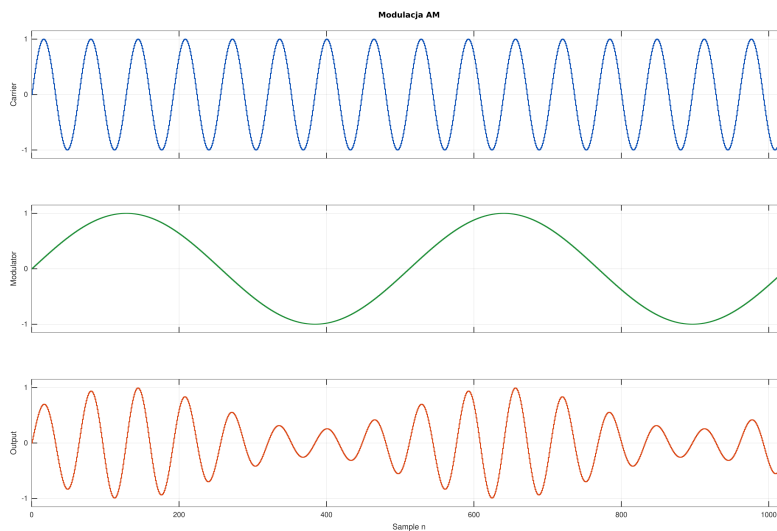
Podczas implementacji przyjęto następujące założenia:

- wszystkie moduły korzystają ze wspólnego pakietu `dds_pkg`,
- typy wyliczeniowe są używane zamiast nieopisanych kodów liczbowych,
- ten sam mechanizm DDS jest wykorzystywany zarówno dla nośnej, jak i modulatora,
- architektura została podzielona na małe moduły o jasno określonych odpowiedzialnościach,
- zastosowano potokowanie w celu uproszczenia spełnienia wymagań czasowych.

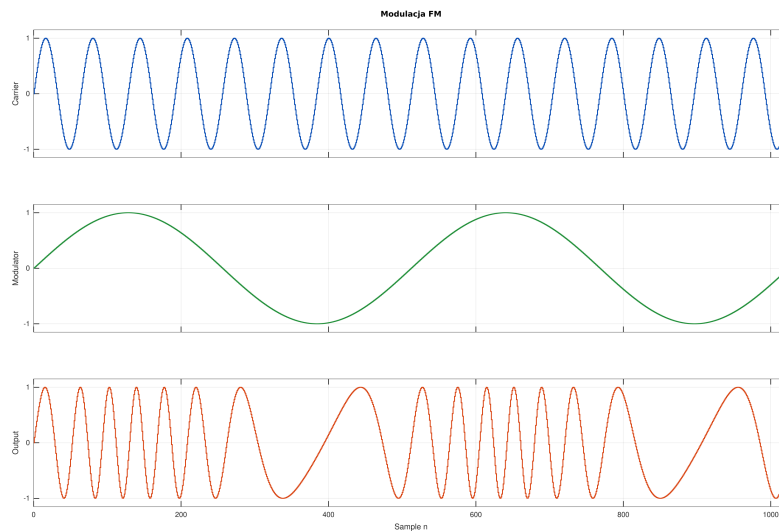
Dodatkowym elementem projektu jest wyrównywanie latencji pomiędzy poszczególnymi ścieżkami sygnałowymi, szczególnie w torze modulacji amplitudy.



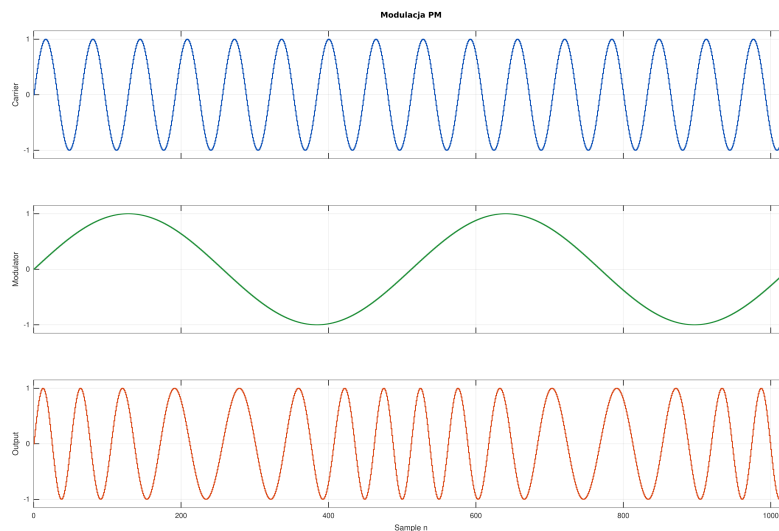
Rysunek 5.1: Przebieg sinusoidalnej nośnej DDS w modelu behawioralnym.



Rysunek 5.2: Przykład modulacji AM w modelu behawioralnym DDS.



Rysunek 5.3: Przykład modulacji FM w modelu behawioralnym DDS.



Rysunek 5.4: Przykład modulacji PM w modelu behawioralnym DDS.

5.7 Ograniczenia implementacji

Obecna wersja projektu posiada kilka ograniczeń:

- szerokość próbki wyjściowej jest ograniczona do `SAMPLE_WIDTH`,
- dokładność generowanego sygnału zależy od liczby iteracji algorytmu CORDIC,
- modulacja FM nie dopuszcza ujemnej efektywnej częstotliwości,
- modulacja AM nie realizuje odwracania fazy sygnału przez ujemne wzmocnienie,
- projekt nie zawiera interfejsu rejestrowego ani magistrali konfiguracyjnej.

Pomimo tych ograniczeń projekt realizuje pełną generację DDS z obsługą modulacji AM, FM i PM oraz stanowi kompletną bazę do dalszej rozbudowy.

6. Opis wykonanych testów

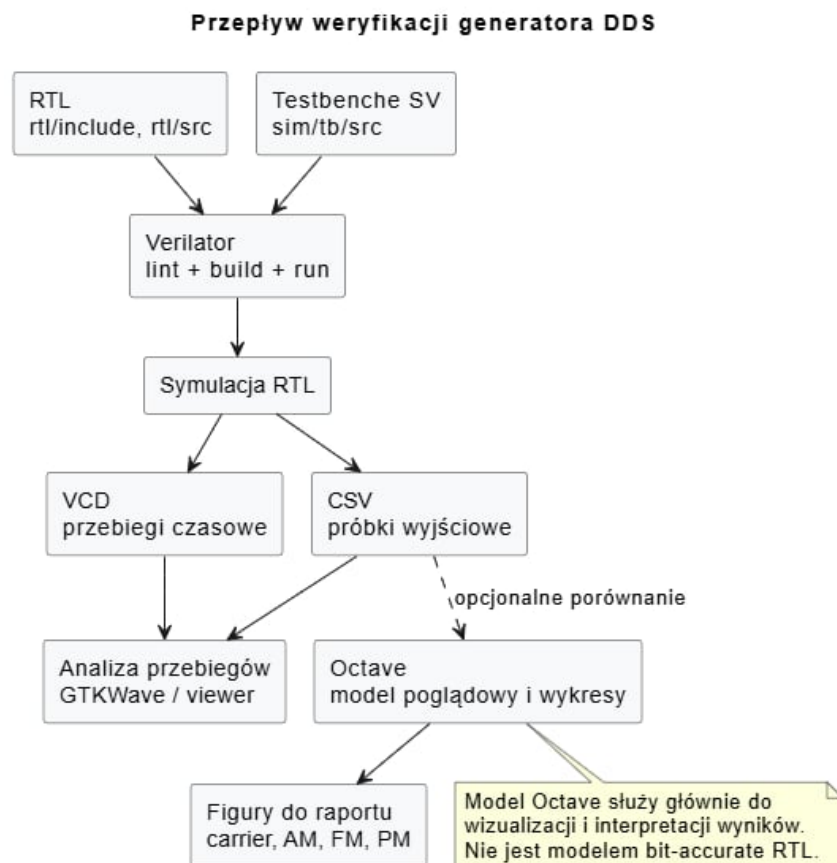
6.1 Cel testów

Celem testów jest potwierdzenie poprawności bloków RTL oraz ich współpracy w `dds_core`. Zakres obejmuje:

- akumulację fazy, offset fazy i sterowanie FTW,
- wybór przebiegów `WAVE_DC`, `WAVE_SINE`, `WAVE_SQUARE`, `WAVE_TRIANGLE`,
- skalowanie amplitudy, zaokrąglenie i saturację,
- tryby `MOD_NONE`, `MOD_AM`, `MOD_FM`, `MOD_PM`,
- pliki VCD i CSV do analizy przebiegów.

6.2 Środowisko testowe

Podstawowym narzędziem testowym jest Verilator uruchamiany przez `scripts/run_verilator.sh`. Testbenche modułowe sprawdzają bloki składowe, natomiast `tb_dds_core` sprawdza integrację między nimi i działanie całego systemu. Wspólne funkcje testowe znajdują się w `sim/tb/include/tb_common_pkg.sv`. Ogólny przepływ weryfikacji projektu pokazuje rysunek 6.1.



Rysunek 6.1: Przepływ weryfikacji i analizy wyników symulacji.

6.3 Testy funkcjonalne

Macierz testów funkcjonalnych znajduje się w tabeli 6.1. Zawiera ona funkcje, metody weryfikacji, oczekiwane zachowanie oraz status testu.

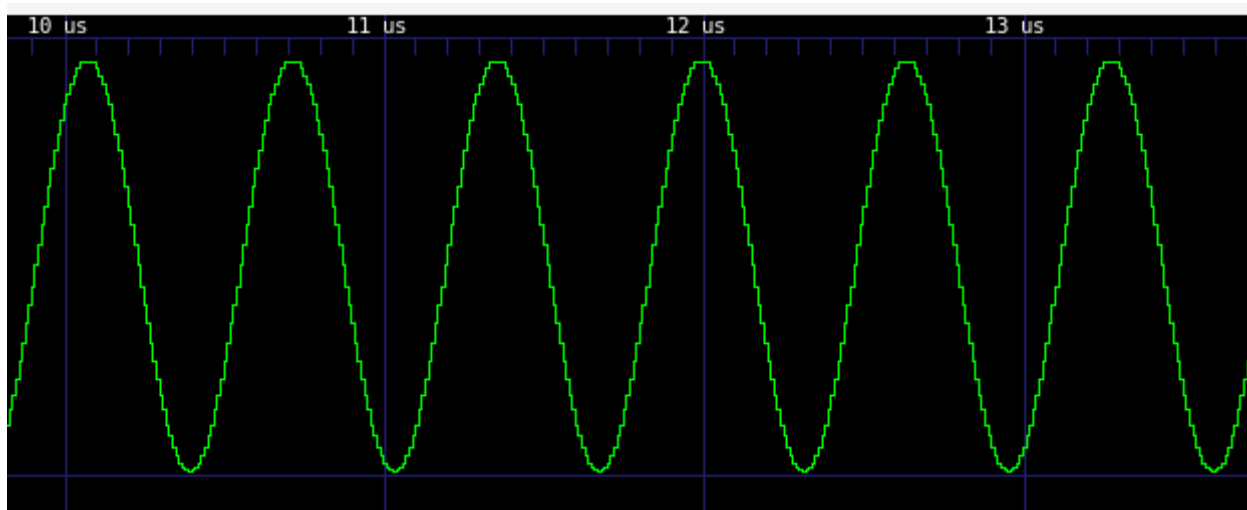
Tablica 6.1: Macierz testów funkcjonalnych

Funkcja	Metoda weryfikacji	Wynik oczekiwany	Status
phase_accumulator	tb_phase_accumulator, Verilator	Faza rośnie o i_ftw przy aktywnym i_en i zawija się modulo $PHASE_WIDTH$.	OK
phase_control	tb_phase_control, Verilator	Faza nośnej uwzględnia $carr_phase$, fm_delta , pm_offset i sygnały ważności.	OK
Generacja przebiegów	tb_waveform_generator, Verilator	$WAVE_DC$, $WAVE_SINE$, $WAVE_SQUARE$ i $WAVE_TRIANGLE$ generują oczekiwane próbki po latencji potoku.	OK
cordic	tb_cordic, Verilator	Sinus i cosinus dla wybranych faz mieszczą się w przyjętej tolerancji.	OK
Skalowanie amplitudy	tb_amplitude_control, Verilator	Tor amplitudy wykonuje mnożenie, zaokrąglanie i saturację do zakresu $SAMPLE_WIDTH$.	OK
Modulacja AM	tb_modulation_engine, tb_dds_core, VCD/CSV	MOD_AM zmienia am_gain i wpływa na amplitudę próbki wyjściowej.	OK
Modulacja FM	tb_modulation_engine, tb_phase_control, tb_dds_core	MOD_FM generuje fm_delta , który zmienia efektywne FTW nośnej bez ujemnej częstotliwości.	OK
Modulacja PM	tb_modulation_engine, tb_phase_control, tb_dds_core	MOD_PM generuje pm_offset , który chwilowo przesuwają fazę nośnej.	OK
Integracja dds_core	tb_dds_core, VCD, CSV	Generator obsługuje tryby MOD_NONE , MOD_AM , MOD_FM , MOD_PM i tworzy próbki oznaczone przez o_sample_valid .	OK

6.4 Analiza przebiegów

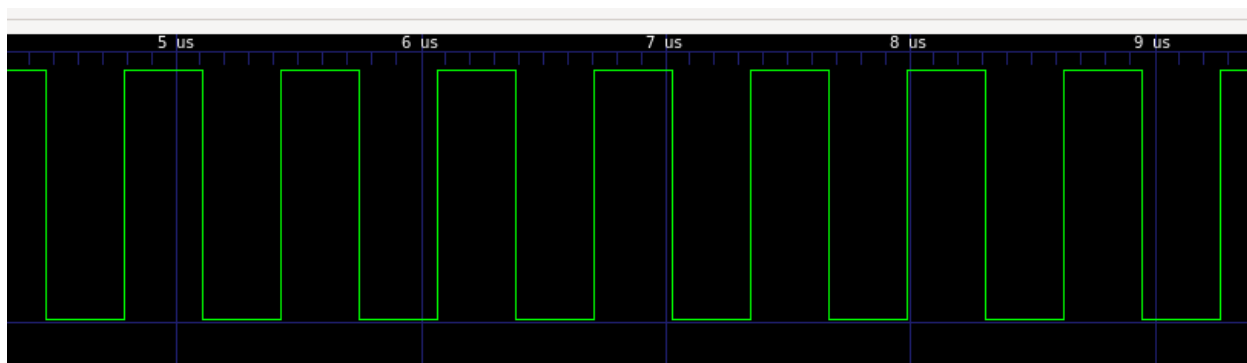
Przebiegi VCD i CSV z `sim/out` służą do wizualnej kontroli trybów pracy generatora DDS.

6.4.1 Przebieg sinusoidalny



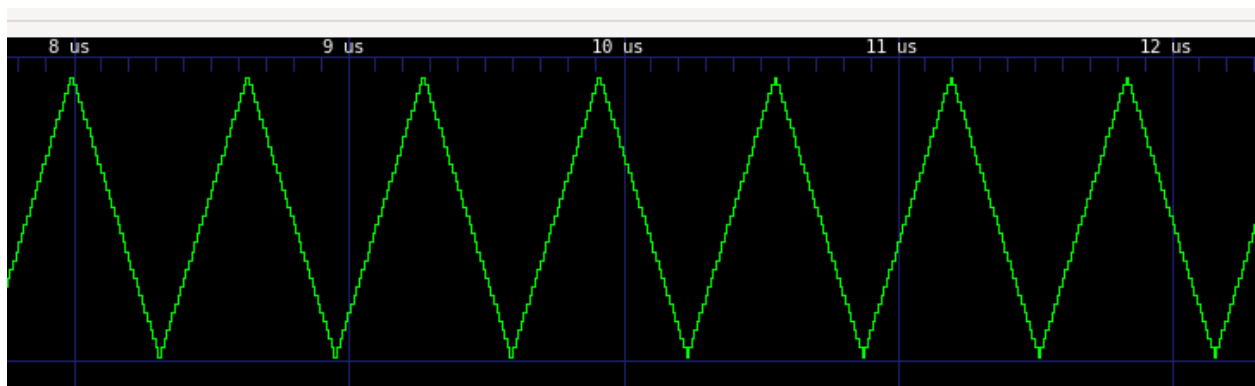
Rysunek 6.2: Przebieg sinusoidalny wygenerowany przez symulację RTL.

6.4.2 Przebieg prostokątny



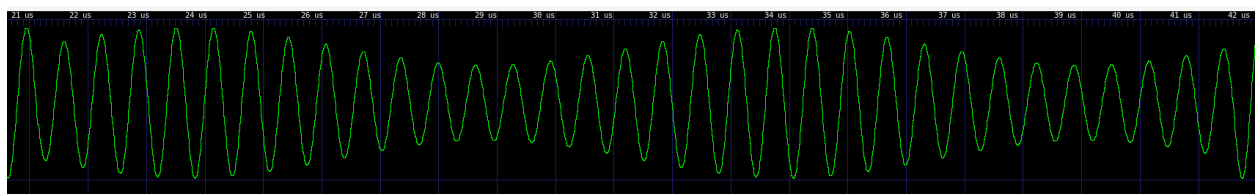
Rysunek 6.3: Przebieg prostokątny wygenerowany przez symulację RTL.

6.4.3 Przebieg trójkątny



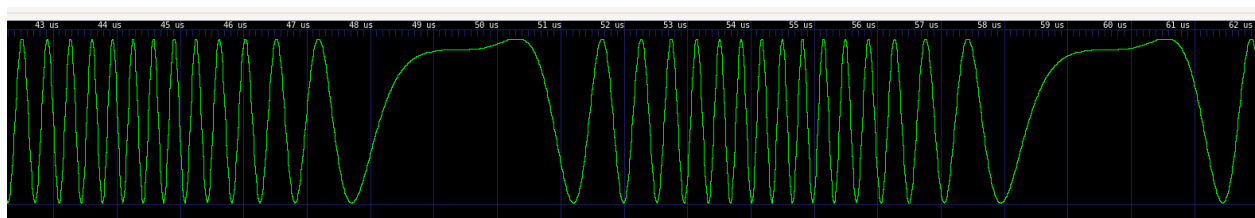
Rysunek 6.4: Przebieg trójkątny wygenerowany przez symulację RTL.

6.4.4 Przebieg AM



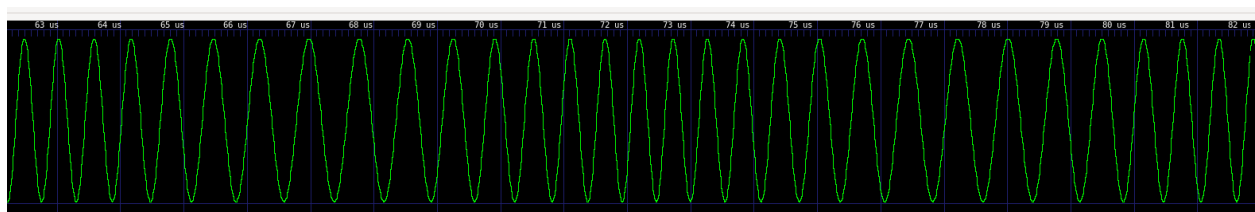
Rysunek 6.5: Przebieg sygnału z modulacją AM wygenerowany przez symulację RTL.

6.4.5 Przebieg FM



Rysunek 6.6: Przebieg sygnału z modulacją FM wygenerowany przez symulację RTL.

6.4.6 Przebieg PM



Rysunek 6.7: Przebieg sygnału z modulacją PM wygenerowany przez symulację RTL.

6.5 Porównanie z modelem Octave

W celu dodatkowej weryfikacji poprawności działania generatora przygotowano model behawioralny w środowisku Octave. Model nie jest zgodny bitowo ani czasowo z implementacją RTL, jednak pozwala na porównanie ogólnego zachowania systemu oraz oczekiwanych efektów modulacji.

Porównanie wyników wykazało zgodność pomiędzy przebiegami generowanymi przez model Octave i implementację RTL. Dotyczy to zarówno podstawowych przebiegów nośnej, jak i działania modulacji AM, FM oraz PM. W obu przypadkach zaobserwowano takie same zależności jakościowe, w szczególności zmiany amplitudy dla AM, chwilowej częstotliwości dla FM oraz przesunięcia fazowego dla PM.

Różnice pomiędzy wynikami wynikają głównie z zastosowania arytmetyki stałoprzecinkowej, ograniczonej szerokości słów oraz latencji potoków obecnych w implementacji sprzętowej. Nie wpływają one jednak na funkcjonalną zgodność otrzymanych rezultatów.

Przykładowe przebiegi wygenerowane przez model Octave przedstawiono na rysunkach [5.1](#), [2.1](#), [5.2](#), [5.3](#) oraz [5.4](#). Uzyskane wyniki są zgodne z obserwacjami pochodzącymi z symulacji RTL, co stanowi dodatkowe potwierdzenie poprawności implementacji generatora DDS.

7. Podręcznik użytkownika

7.1 Wymagane narzędzia

Wymagane narzędzia:

- Verilator – lint, budowanie i uruchamianie testbenchy SystemVerilog,
- GNU Octave – uruchamianie modelu behawioralnego i generowanie figur.

7.2 Struktura projektu

Najważniejsze katalogi:

- rtl/include – pakiet dds_pkg.sv z typami, parametrami i funkcjami pomocniczymi,
- rtl/src – moduły syntetyzowalne RTL,
- sim/tb/include – wspólne elementy testbenchy,
- sim/tb/src – testbenche modułowe i integracyjne,
- sim/out – katalog tworzony przez symulacje na pliki VCD i CSV,
- model/octave – model behawioralny DDS i generator wykresów,
- model/octave/figures – domyślne figury SVG generowane z modelu Octave,
- scripts – skrypty pomocnicze do Verilator i Octave.

7.3 Symulacja z użyciem Verilator

scripts/run_verilator.sh obsługuje lint, budowanie i uruchamianie testbenchy.

```
1 scripts/run_verilator.sh lint
2 scripts/run_verilator.sh build
3 scripts/run_verilator.sh run
4 scripts/run_verilator.sh all
5 scripts/run_verilator.sh clean
```

Listing 7.1: Podstawowe komendy Verilator.

Zalecany przepływ:

1. uruchomić scripts/run_verilator.sh lint po zmianach w RTL,
2. uruchomić scripts/run_verilator.sh all przed uznaniem zmiany za gotową,
3. sprawdzić komunikaty testbenchy w terminalu,
4. w razie potrzeby otworzyć pliki VCD z katalogu sim/out.

tb_dds_core dodatkowo tworzy sim/out/tb_dds_core_trace.csv. Pliki VCD można oglądać w GTK-Wave albo w przeglądarce przebiegów środowiska symulacyjnego.

7.4 Użycie modelu Octave

Model Octave znajduje się w `model/octave`. Figury SVG i wersje PNG do dokumentacji generuje:

```
1 scripts/run_octave.sh all
```

Listing 7.2: Generowanie figur Octave i konwersja do PNG z katalogu głównego repozytorium.

Alternatywnie można uruchomić model bezpośrednio:

```
1 cd model/octave
2 octave --no-gui run_demo.m
```

Listing 7.3: Bezpośrednie uruchomienie modelu Octave.

`run_demo.m` zapisuje SVG w `model/octave/figures`. `scripts/svg_to_png.sh` konwertuje je przez Inkscape do `docs/figures/*.png`.

7.5 Użycie w Vivado

Repozytorium nie zawiera gotowego projektu Vivado ani skryptu TCL. Minimalny przepływ ręczny:

1. utworzyć nowy projekt Vivado dla docelowej płytki lub układu FPGA,
2. dodać plik `rtl/include/dds_pkg.sv`,
3. dodać moduły z katalogu `rtl/src`,
4. ustawić `dds_core_wrapper` jako top dla integracji z płaskimi portami albo `dds_core` jako top dla pracy ze strukturą SystemVerilog,
5. dodać wybrane testbenche z `sim/tb/src` do zbioru plików symulacyjnych.

7.6 Konfiguracja generatora DDS

Przy integracji przez `dds_core` wartości są przekazywane w `dds_config_t`; przy `dds_core_wrapper` jako osobne porty. Typowy proces:

1. ustawić `en` na zero podczas przygotowania konfiguracji,
2. dobrać `carr_ftw` do wymaganej częstotliwości nośnej,
3. ustawić `carr_phase`, `carr_amp` i `carr_wave`,
4. dla pracy bez modulacji wybrać `MOD_NONE`,
5. dla modulacji ustawić `mod_type`, `mod_ftw`, `mod_depth` oraz `mod_wave`,
6. aktywować generator przez `en`,
7. odbierać tylko próbki oznaczone przez `o_sample_valid`.

Przykłady bez narzucania konkretnego zegara przedstawia tabela 7.1.

Tablica 7.1: Przykładowe konfiguracje użytkowe generatora

Cel	Ustawienia
Nośna sinusoidalna	<code>carr_wave = WAVE_SINE, mod_type = MOD_NONE.</code>
Przebieg prostokątny	<code>carr_wave = WAVE_SQUARE, mod_type = MOD_NONE.</code>
Modulacja AM	<code>mod_type = MOD_AM, ustawić mod_ftw, mod_depth i mod_wave.</code>
Modulacja FM	<code>mod_type = MOD_FM, ustawić częstotliwość i głębokość sygnału modulującego.</code>
Modulacja PM	<code>mod_type = MOD_PM, ustawić sygnał modulujący jako zmienny offset fazy.</code>

8. Metodologia rozwoju i utrzymania systemu

8.1 Organizacja projektu

Projekt został podzielony na kilka głównych części:

- `rtl` – implementacja modułów w języku SystemVerilog,
- `sim` – testbenche oraz pliki związane z symulacją,
- `model/octave` – model behawioralny i skrypty generujące wykresy,
- `docs` – dokumentacja projektu.

Taki podział umożliwia niezależny rozwój implementacji sprzętowej, środowiska testowego oraz materiałów pomocniczych.

8.2 Metodologia weryfikacji

Podstawowym narzędziem weryfikacji RTL był Verilator. Testbenche sprawdzały poprawność działania generatora dla różnych konfiguracji nośnej oraz modulacji.

Proces weryfikacji obejmował:

- analizę statyczną kodu (lint),
- uruchamianie testbenchów modułowych,
- analizę przebiegów czasowych VCD,
- porównanie wyników z modelem referencyjnym.

Dodatkowo projekt został przygotowany do dalszej weryfikacji w środowisku Vivado, obejmującej symulację behawioralną, syntezę oraz analizę wykorzystania zasobów FPGA.

8.3 Model referencyjny Octave

W ramach projektu przygotowano model behawioralny w środowisku Octave. Jego zadaniem było przedstawienie oczekiwanego zachowania generatora DDS oraz wizualizacja efektów działania modulacji AM, FM i PM.

Model wykorzystuje arytmetykę zmiennoprzecinkową i nie jest zgodny bitowo ani czasowo z implementacją RTL. Stanowi jednak wygodne narzędzie do analizy działania algorytmów oraz interpretacji wyników uzyskanych podczas symulacji sprzętowej.

Porównanie wyników modelu i implementacji RTL wykazało zgodność funkcjonalną generowanych przebiegów oraz efektów modulacji.

8.4 Możliwości dalszego rozwoju

Obecna implementacja stanowi kompletny generator DDS z obsługą modulacji AM, FM i PM, jednak może zostać rozszerzona o dodatkowe funkcjonalności.

Przykładowe kierunki dalszego rozwoju obejmują:

- dodanie interfejsu rejestrowego do konfiguracji z poziomu procesora,
- integrację z magistralą AXI Lite,
- zwiększenie szerokości próbki wyjściowej,
- analizę widmową generowanych sygnałów,
- implementację i demonstrację projektu na docelowej platformie FPGA.

Bibliografia

- [1] National Instruments. Understanding direct digital synthesis (dds), 2025. Online. Dostęp: 2025-06-08. <https://www.ni.com/en/shop/electronic-test-instrumentation/waveform-generators/understanding-direct-digital-synthesis--dds-.html>.
- [2] Stuart Sutherland and Don Mills. Synthesizable systemverilog: Busting the myth that systemverilog is only for verification, 2013. SNUG Silicon Valley. https://sutherland-hdl.com/papers/2013-SNUG-SV_Synthesizable-SystemVerilog_paper.pdf.
- [3] Verilator Project. Verilator user guide, 2025. Online. Dostęp: 2025-06-08. <https://verilator.org/guide/latest/>.